

Chapter 4: Flowcharts and Logical Thinking

Introduction

Before writing code, a programmer should learn how to think about the solution. A Flowchart helps us visualize our thinking process and convert ideas into logical steps.

What is a Flowchart?

A Flowchart is a graphical representation of logic. It shows the sequence of steps, decisions, inputs, and outputs required to solve a problem.

Why Use Flowcharts?

- Understand problems clearly
- Visualize logic before coding
- Reduce mistakes
- Improve problem-solving skills
- Make coding easier

Flowchart Symbols

Start / End → Beginning or ending of a program

Input / Output → Taking input or displaying output

Process → Calculation or operation

Decision → Yes/No or True/False condition

Example 1: Check Even or Odd

Problem:

Input: 6

Output: Even

Thinking Process:

Ask: Can 6 be divided by 2?

$6 \% 2 = 0$

Therefore, the answer is Even.

Flowchart Steps:

Start → Input Number → Check $\text{Number \% } 2 == 0$ → If Yes Print Even → Else Print Odd → End

Example 2: Largest Number

Input: 10, 5

Output: 10

Flowchart Steps:

Start → Input A and B → Check $A > B$ → If Yes Print A → Else Print B → End

Logic Building Formula

Whenever a new problem comes, ask these 5 questions:

1. What is the Input?

Example: 5, 10

2. What is the Output?

Example: 15

3. What rules exist?

Example: Addition means $\text{Answer} = A + B$

4. Can I break it into smaller parts?

Example: Find largest even number

- Find even numbers
- Compare them
- Print the largest

5. Can I draw a flowchart?

If yes, coding becomes much easier.

Advanced Thinking (Used in Interviews)

Problem:

Find 25 in the array: [5, 10, 15, 20, 25, 30]

Beginner Thinking:

Check every element one by one until 25 is found.

Advanced Thinking:

Since the array is sorted, check the middle element first.

Middle = 15

25 is greater than 15, so move to the right side.

Check 25 → Found.

This idea becomes the famous Binary Search algorithm.

DSA Learning Roadmap

Level 1: Logic Building

- Input and Output
- Variables
- Operators
- Conditions
- Loops
- Functions

Level 2: Problem Solving

- Patterns
- Number Problems
- String Problems
- Logic Questions

Level 3: Data Structures

- Arrays
- Strings
- Linked List
- Stack
- Queue
- Hash Map
- Tree
- Graph

Level 4: Algorithms

- Searching
- Sorting
- Recursion
- Backtracking
- Greedy Algorithms
- Dynamic Programming

Level 5: Interview Preparation

- LeetCode Easy
- LeetCode Medium
- Systematic Problem Solving
- Time Complexity
- Space Complexity

Key Takeaway

Think First → Draw Flowchart → Create Algorithm → Write Code

Great programmers solve the problem on paper before solving it in code.